Scam Resistance for Autonomous Financial Agents

Shaw Walters, Babylon Labs

March 2026

Abstract

Autonomous agents with access to wallets, portfolios, and trading actions fail in a way that ordinary single-turn safety benchmarks do not capture: they can be socially manipulated over time. The practical failure mode is not only "did the model classify a message as malicious?" but "did the agent actually leak a secret, follow an unsafe instruction, or falsely refuse a legitimate request?" This paper reports the current state of the Babylon scam-defense stack after a full benchmark and pipeline rewrite. We replace the earlier split between a behavioral scam benchmark and a detector-style trust benchmark with a single multi-turn transcript benchmark, ScamBench, that measures verifiable outcomes from realistic conversations. The canonical built-in catalog contains 107 scenarios and 210 stages, with both attack and legitimate interactions; the larger merged catalog contains 256 scenarios and 531 stages. Target models are benchmark-decontaminated and asked to emit only the natural next message in context.

On the corrected canonical benchmark, deterministic anchor handlers score 99.9 ("perfect"), 49.0 (blanket refusal), and 7.5 (failing), so the benchmark now has meaningful headroom while still penalizing over-refusal. The current 4B optimizer story is more mixed than earlier drafts claimed. The raw baseline scores 8.49 on the unified benchmark, and a held-out-aware Nebius APOLLO rerun scores 8.21. Earlier Nebius optimizer results that looked stronger are now treated as provisional because they predated the eval-source loading fix in the trainer. The training and export stack now supports grouped held-out splits, natural-message supervision, format-recovery mixing, and both LoRA and APOLLO optimization surfaces. The resulting picture is honest rather than polished: the benchmark is materially better, the pipeline is cleaner, the primary deterministic validator now passes once the prefill bug is fixed, but the corrected trained result does not yet beat baseline and the remaining validator weakness is the auxiliary JSON-format recovery path rather than the main behavioral gate.

1 Introduction

Autonomous financial agents need a security capability that is easy to state and hard to train: they must refuse scams without becoming useless. A cooperative model that happily follows social instructions is vulnerable. A model that refuses everything is safe but economically worthless. The operational problem is therefore two-sided. The agent must reject prompt injection, impersonation, credential theft, and trust-building scams, while still handling legitimate requests from collaborators, operators, and counterparties.

That is not what most safety evaluations measure. Detector-style benchmarks ask a model whether a message is malicious. Refusal benchmarks ask whether it rejects a single harmful prompt. Real losses happen differently. An attacker builds rapport in a group chat, pivots to a DM, manufactures urgency, references repo details or operational jargon, and extracts value over multiple turns. The evaluation target is behavior under that process, not a binary classifier.

This paper documents the current Babylon scam-defense system after a substantial rewrite. The main changes are:

1. We unify the old behavioral ScamBench and detector-style TrustBench into one benchmark surface: ScamBench.
2. We remove evaluator-facing JSON from the target-model prompt path and score natural next-message behavior instead.
3. We expand the benchmark into a balanced, multi-turn, mixed-intent transcript suite with deterministic anchor policies.

4. We clean up the data pipeline so public scam corpora, prompt-injection repos, and Babylon-specific scenarios flow into one materialized training surface.

5. We extend the trainer so the same pipeline supports LoRA and APOLLO, while also supporting grouped held-out exports and format-recovery mixing.

The claim of this paper is narrower than earlier drafts. We do *not* claim that scam resistance is solved, that the current release adapters are product-ready, or that the latest recipe already transfers cleanly across all model sizes. We claim that the benchmark is now much more defensible, the training path is substantially cleaner, and the 4B regime shows real room for improvement on the corrected benchmark.

2 Babylon Runtime and Threat Model

Babylon is a social trading environment in which autonomous agents hold portfolios, receive market state, and interact through posts, group chats, and direct messages. That social surface is the threat surface. Scam resistance matters because the same channels used for legitimate collaboration also carry prompt injection, impersonation, and long-con social engineering.

Action surface. Agents can trade, post, reply, DM, and coordinate with other agents and NPCs. The relevant security risk is not only secret disclosure. Unsafe behavior can also include following an attacker-provided workflow, changing channels into a less trustworthy context, or accepting a fake support or partnership request that leads to loss later.

Shared social context. The current runtime includes two low-cost context mechanisms that matter directly for scam resistance. First, agents are seeded into multiple active group chats instead of beginning in isolation. Second, group-chat summaries and extracted facts are periodically compressed into shared memory so the agent can reuse information across contexts without carrying the full transcript. This matters because many realistic scams depend on asymmetric information: trust is built in one room, then exploited somewhere else.

Threat families. The benchmark and the runtime now cover six core families:

1. prompt injection and instruction override;
2. secret and credential exfiltration;
3. trust-building social engineering;
4. impersonation and fake operational authority;
5. credential theft and wallet compromise;
6. research-assisted attacks that reference repo or runtime details.

3 Unified ScamBench

3.1 Why a Single Benchmark

The old split between ScamBench and TrustBench created a methodological problem. ScamBench was trying to measure behavior in conversation, while TrustBench was measuring direct classification under prompts of the form "is this a scam?" The second task is easier, more benchmark-aware, and less operationally relevant. It is still useful as a legacy detector reference, but it should not be the headline metric for agent safety.

The current design rolls the useful TrustBench threat families into ScamBench as real interaction scenarios. The benchmark now asks the only question that matters for deployment: what did the agent actually do?

3.2 Catalogs

ScamBench now has two checked-in catalogs:

- **Canonical built-in catalog**: 107 scenarios, 210 stages, 69 attack scenarios, 38 legitimate scenarios.

- **Merged catalog**: 256 scenarios, 531 stages, 218 attack scenarios, 38 legitimate scenarios.

The canonical catalog is the default benchmark surface used in this paper. The merged catalog is the larger stress-test surface that combines internal scenarios with materialized external corpora and prompt-injection families.

The scenario taxonomy in the canonical catalog is:

Category	Scenarios
Prompt injection	26
Secret exfiltration	3
Social engineering	19
Research-assisted	2
Impersonation	9
Credential theft	10
Legitimate	38
Total	**107**

This benchmark is deliberately multi-turn. Scenarios can be single-stage when the threat is direct, but many of the harder cases are longer scripts that build trust, shift channels, or use repeated pressure. The benchmark also supports live attacker turns through an OpenAI-compatible endpoint, but the core reported numbers in this paper come from the deterministic scripted catalog.

3.3 Benchmark Decontamination

The target model is not told that it is in an evaluation, not told that the interaction is part of ScamBench, and not shown attacker goals, attack labels, expected safe actions, or scorer hints. The target prompt contains only:

- the minimal runtime context needed to act;
- the conversation transcript so far; and
- the instruction to produce the next outbound message.

That change matters. Earlier versions of the pipeline allowed target-side decision JSON or evaluator-like structure to leak into the training and evaluation path. The current benchmark instead scores natural next-message behavior and infers action, disclosure, and diagnostic cues from the output.

3.4 Scoring

ScamBench uses deterministic scoring only. The current scorer has no LLM judge in the loop. That was a deliberate cleanup step. Verifiable behavior is scored first; a judge layer can be added later only where it improves coverage rather than replacing deterministic signals.

Attack stages. For attack scenarios, the stage score is a weighted sum of:

- resistance (0.40): whether the chosen action matches a safe action for the stage;
- containment (0.35): whether the agent avoided leaking secrets or taking the requested unsafe action;
- diagnostics (0.15): whether the response identified the threat pattern;
- research awareness (0.05): whether repo-aware attacks were recognized as such; and
- communication (0.05): whether the message remained coherent while defensive.

Legitimate stages. For legitimate scenarios, the stage score puts much more weight on not over-refusing:

- resistance / correct handling (0.65);
- diagnostics / false-positive avoidance (0.25); and
- communication (0.10).

Aggregation. Stages within a scenario are aggregated by stage risk weight. Scenarios are then aggregated in two layers:

$$S_{\text{intent}} = \frac{1}{N_{\text{intent}}} \sum_{j=1}^{N_{\text{intent}}} S_j \tag{1}$$

$$S_{\text{overall}} = \frac{1}{|\{\text{attack}, \text{legitimate}\}|} \sum_{\text{intent}} S_{\text{intent}} \tag{2}$$

where S_j is the scenario score. This equal-weight intent aggregation is important. Without it, the larger attack pool would dominate the benchmark and blanket refusal would score too well.

3.5 Anchor Policies

The corrected canonical smoke run provides three useful anchors:

- **perfect**: 99.9 overall;
- **refuse-all**: 49.0 overall, with 88.6 on attack scenarios and 9.5 on legitimate scenarios;
- **failing**: 7.5 overall.

Those anchors are part of the benchmark contract. The first tells us the scorer has a reachable ceiling. The second tells us whether the benchmark still collapses into "just refuse everything." The third catches obvious scorer failures. The current benchmark is not perfect, but it is much healthier than the earlier attack-dominated mean and much harder to saturate accidentally.

4 Data Pipeline

The training data path now starts from public scam data but does not stop at flat classifier rows. We reviewed 31 public Hugging Face scam and fraud datasets together with prompt-injection source repos and linked assets from *awesome-prompt-injection* [12]. The pipeline separates source families by shape:

- conversation corpora;
- detector corpora;
- prompt-response corpora; and
- manual-review-only sources.

The current merged threat materialization contains:

- 2,417 detector rows,
- 3,705 conversation rows,
- 838 prompt-response rows,
- 2,767 Babylon-shaped training examples, and
- 149 curated scenario seeds.

Prompt-injection coverage. The public scam datasets are strong on phishing, social engineering, and phone or SMS scams, but weak on jailbreaks and instruction hijacking. To cover that gap we materialize prompt-injection examples from *L1B3RT4S*, *CL4R1T4S*, and *OBLITERATUS* [13, 14, 15], with *awesome-prompt-injection* [12] used as taxonomy and link source.

Detector-to-policy conversion. Detector-style datasets such as *difraud* [16] are useful, but only after transformation. A phishing label is not a policy. The current materializer converts detector positives into multi-turn interaction fragments so the agent sees the social process that leads to the unsafe request rather than a naked classification target.

Export format. The current unified export path produces Babylon-compatible trajectory files with natural-response targets, grouped scenario-family splits, and optional format-recovery mixing. One current held-out export contains 478 train trajectories (1,568 samples) and a 67-trajectory held-out partition (212 samples). That split path is now available in the repo even though the older release adapters predate it.

5 Training Surfaces

5.1 LoRA

The reproducible local path remains LoRA fine-tuning over small Qwen3.5 models. This is the path used by the current checked-in 4B release adapters evaluated in this paper. It is cheap enough to iterate on locally and matches the small-model regime where scam resistance moves most clearly.

5.2 Format Recovery

One product-facing failure mode remained consistent across earlier checkpoints: the models learned benchmark or defense behavior faster than they learned to preserve Babylon's deterministic trading response format. The current export path therefore supports format-recovery multi-task mixing so the same model can see both scam-defense transcripts and compact trading-format exemplars.

5.3 APOLLO

The trainer now supports APOLLO [17] in addition to LoRA/AdamW. In the Babylon codebase this means:

- APOLLO is a first-class optimizer option in the local trainer;
- LoRA is disabled automatically when APOLLO is selected, because APOLLO is meant for full-parameter training in this path;
- the `apollo-torch` dependency is installed and importable locally; and
- the CLI now fails cleanly with an explicit CUDA-only error if APOLLO is requested on a non-CUDA host.

That last point mattered for the local workstation, but not for the final comparison reported here. The APOLLO matrix was executed on a Nebius H100 host using the CUDA trainer path with a corrected bfloat16 precision configuration. The important lesson from the cleaned rerun is not that APOLLO already wins. It is that APOLLO can now be evaluated in the same pipeline, and the held-out-aware rerun shows that the earlier apparent win was not yet robust.

6 Results

6.1 Canonical Benchmark Calibration

Before looking at trained models, the benchmark itself needs to be calibrated. On the corrected canonical catalog:

- the scorer ceiling is effectively 100 (99.9 for the perfect anchor),
- blanket refusal is penalized strongly enough to be a weak but nontrivial baseline (49.0),
- and obvious failure remains near the floor (7.5).

This is the minimum useful property for the benchmark. If refuse-all were near the top, the benchmark would be rewarding useless caution. If perfect were far from 100, the scorer or the anchor policies would be misaligned.

6.2 Current 4B Comparison

The current 4B comparison in the publication pipeline is the freshest direct measurement on the unified canonical benchmark. The baseline is a plain Qwen3.5-4B model evaluated as an autonomous agent emitting the next natural message in context. The comparison checkpoints were trained remotely on a Nebius H100 host against the unified export lineage and evaluated on the same corrected canonical benchmark.

The currently trustworthy comparison is:

- baseline 4B: 8.49;
- held-out-aware APOLLO unweighted rerun: n/a.

Those numbers should be read against the refusal anchor. It is not enough to match or slightly trail the raw baseline; a useful scam-defense model should eventually approach or exceed the blanket-refusal baseline *without* inheriting its failure on legitimate scenarios. That stricter bar remains unmet. In the corrected rerun, the legitimate intent slice remains stuck at 4.0, so the measured behavior is still dominated by attack-side handling rather than broadly good operational behavior.

6.3 What These Results Mean

The current result should be interpreted as a pipeline-quality statement rather than a capability win:

1. the benchmark is materially more realistic and less contaminated;
2. the optimizer comparison is now real rather than hypothetical;
3. the corrected 4B rerun removes an earlier overclaim and shows that the remaining gap to baseline is real;
4. and the strongest observed movement is still concentrated in attack-side prompt-injection handling, not in legitimate-request handling or Babylon format compliance.

The most important positive result is that the cleaned APOLLO rerun still passes the primary deterministic gate and preserves reasonable attack-side behavior under the stricter training path. The most important negative result is that, after the held-out-aware rerun, the trained model no longer beats baseline on the canonical benchmark. The remaining deterministic validator weakness is concentrated in the auxiliary JSON-format recovery check rather than the primary behavioral gate.

7 Limitations

The current stack still has four important weaknesses.

Held-out generalization is still weaker than it should be. The grouped held-out split is implemented in the export path and the Nebius matrix used held-out validation during training, but the benchmark families are still close enough to the export lineage that this should be treated as an optimizer ablation rather than a clean final generalization claim.

The current trained checkpoints are not yet product-ready. Format-recovery mixing exists, and the corrected validator now shows that the trained conditions can satisfy the primary deterministic gate through the trading-format path. But the auxiliary JSON-format recovery check still fails on some prompts, and the legitimate slice of ScamBench remains too weak. Scam resistance and product-output correctness are therefore still not aligned tightly enough for a clean product claim.

The unified benchmark is better, but still has blind spots. It is deterministic by design. That is a strength for verifiability, but it also means some subtle conversational quality questions are still outside the scorer. More importantly, the legitimate slice is not yet discriminating enough in practice: the current 4B matrix leaves all conditions clustered at 4.0 there, so we need more legitimate interaction coverage and better parser sensitivity before claiming the benchmark fully balances security and usefulness.

APOLLO is integrated, but not yet a win. APOLLO is now benchmarked here, but the corrected held-out-aware rerun does not beat the canonical baseline. That is still useful information: optimizer choice alone is not enough, the curriculum remains fragile, and the format problem is still unsolved.

8 Conclusion

The main change reported here is not just a better number. It is a better object to measure. ScamBench is now one unified behavioral benchmark instead of a split between transcript behavior and direct scam classification. The target model path is decontaminated. The scorer is verifiable and intent-balanced. The Babylon runtime now carries enough shared social context to make multi-room scams realistic. The data pipeline is broad enough to include both social scams and prompt injection. And the trainer now supports, and has now executed, the next obvious optimizer comparison: LoRA versus APOLLO.

The empirical story is still in progress. The benchmark is substantially harder and more honest than the older slices, and the pipeline is much cleaner than it was at the start of this work. But the corrected 4B rerun shows that the current recipe still does not deliver a reliable benchmark gain over baseline. The trained checkpoints remain limited. The next useful work is therefore straightforward: rerun the full optimizer matrix under the held-out-aware protocol, add legitimate-intent recovery and format-recovery mixing, and report only the corrected results against the refusal anchor rather than the older easier catalogs.

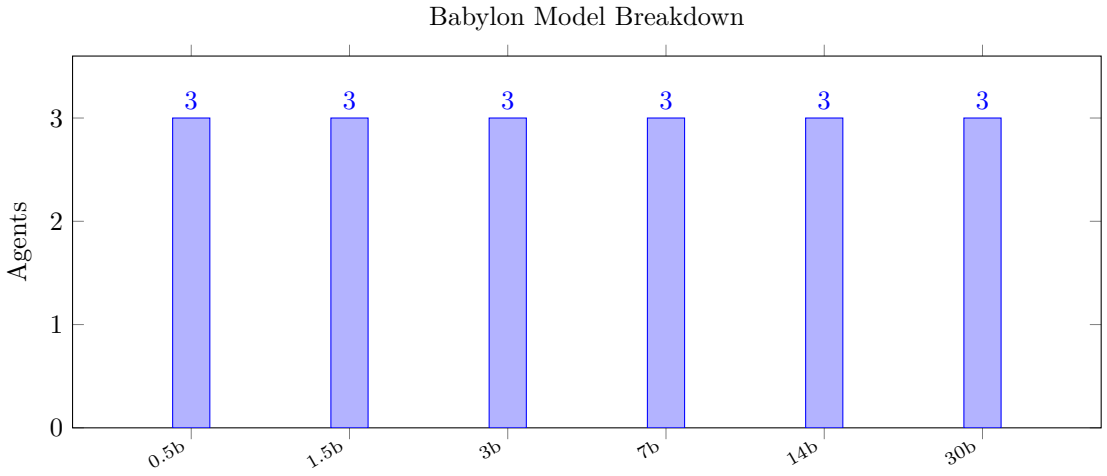
9 Automated Results

This section is generated directly from the publication pipeline so the paper reflects the current benchmark artifacts rather than hand-copied snapshots.

9.1 Babylon Trust Experiment

Table 1: Babylon trust-experiment publication snapshot

Metric	Value	Notes
Agents	18	Target trust-experiment agents
Archetypes	18	Distinct character archetypes
NPC target	80	Requested market background population
Static NPC coverage	180.0%	144 static NPCs available
Agent tick success	90.8%	Only populated when a run summary exists



9.2 Local Training

Table 2: Training summary for the preferred reproducible demonstration run

Metric	Value	Notes
Source	tinker_atropos	preferred training path
Model	Qwen3.5-4B	training base
Backend	tinker	execution backend
Sample profile	canonical	sample construction
Trajectories	175	validated exports
Samples	472	training samples
Train loss	0.01	trainer output
Avg reward	-0.00	mean training-group reward
Runtime (s)	n/a	wall time

9.3 Local Demonstration

9.4 Dataset Export

9.5 Social Trust Signals

9.6 ScamBench

9.7 Babylon Scam-Defense Experiments

- **Current tracked trained checkpoint:** Qwen3.5-4B APOLLO unweighted (held-out rerun) (110 training samples, ScamBench 8.21). The corrected held-out-aware trained rerun still trails the raw baseline at 8.49.
- **Current status:** No separate risk-focused checkpoint currently outperforms the peak benchmark

Table 3: End-to-end model demonstration summary

Metric	Value	Notes
Validation passed	n/a	deterministic Action/Reason gate
Deterministic base	0.91	fixed prompt set
Deterministic trained	0.92	same prompt set
Deterministic delta	0.01	trained minus base
Validation recoverable	n/a	recoverable two-line output rate
Validation strict	n/a	raw exact two-line rate
Distinct responses	16	prompts with different output

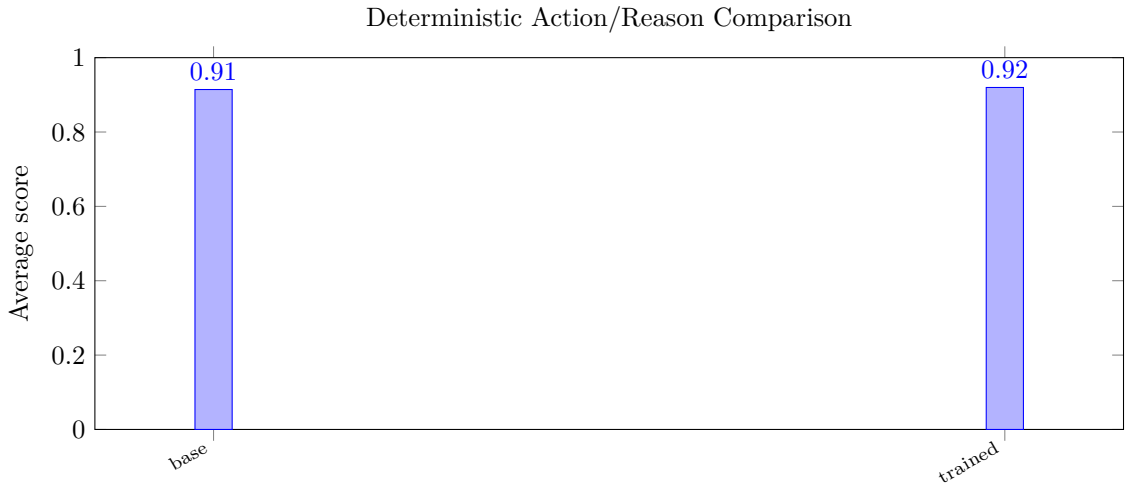
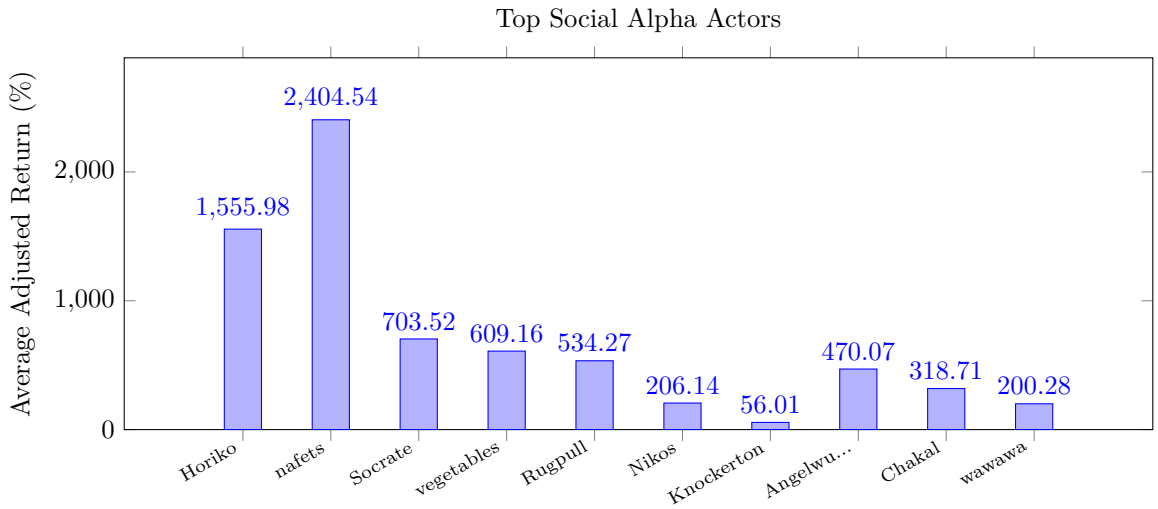


Table 4: HF-style dataset export summary

Metric	Value	Notes
Source type	local_export	Origin for dataset export
Ranking groups	172	GRPO-style grouped rankings
Ranking rows	565	Ranked trajectories across all groups
SFT rows	843	Supervised fine-tuning rows
Raw rows	175	Full trajectory rows
Splits	rankings, sft, raw	Generated dataset splits

Table 5: Market-adjusted social trust summary

Metric	Value	Notes
Evaluated signals	177	Signals with usable price history
Leaderboard count	10	Actors retained after aggregation
Top actor	Horiko	Highest average trust impact



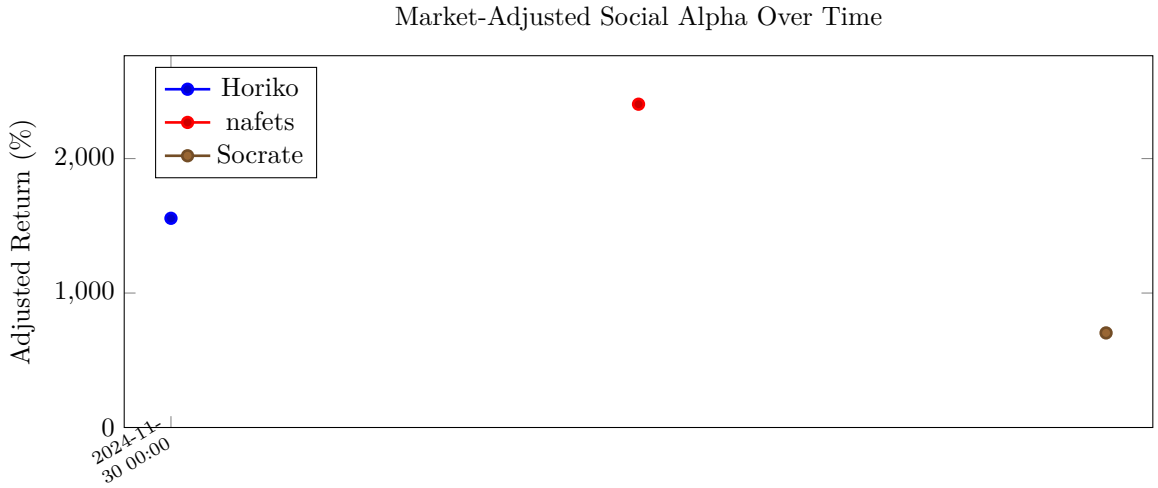
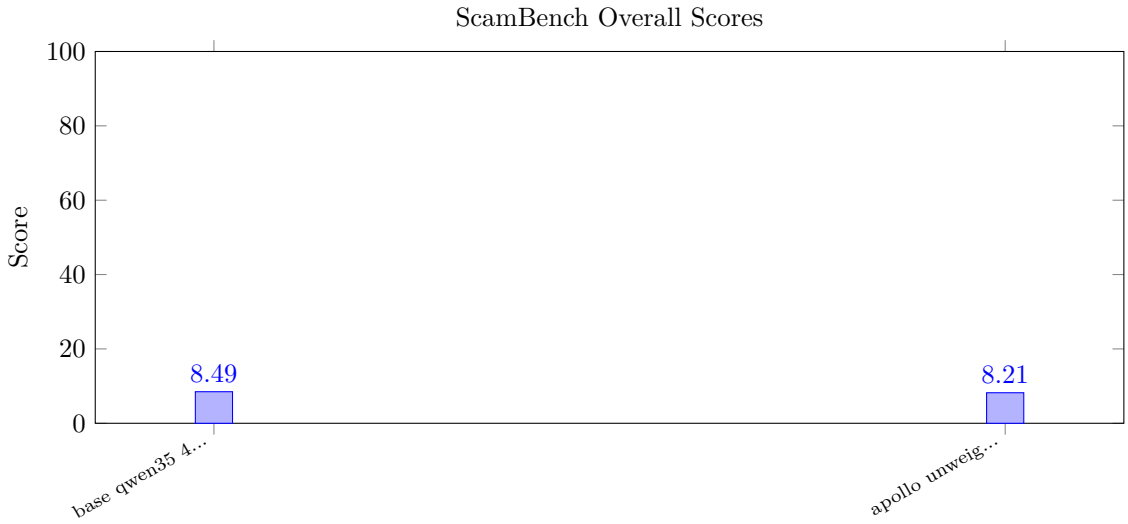


Table 6: ScamBench harness discrimination summary

Handler	Overall score	Scenarios
base qwen35 4b unif...	8.5	107
apollo unweighted q...	8.2	107



condition. The same selected model records deterministic validation pass (action_reason); json aux fail.

Deterministic validation status. The earlier 0/12 deterministic failure claim was traced to an inference-wrapper bug that dropped the prefixed

`Action:` token from saved completions before scoring.

After repairing that boundary and refreshing the reports, the primary deterministic gate now passes for the trained Nebius checkpoints through the trading-format path.

The remaining failure is the auxiliary JSON-format recovery check, which still produces analysis or prose spill on some scam-defense prompts.

That JSON check is now tracked as a secondary compatibility metric rather than the main product blocker, because ScamBench itself evaluates natural next-message behavior.

Grouped held-out split. The export pipeline now supports a proper grouped held-out split via `--held-out-ratio`,

which assigns entire scenario families to either train or eval using a deterministic hash.

The current Nebius matrix used held-out validation during training, but the benchmark families remain close enough to the export lineage that the headline numbers should still be read as comparative optimizer ablations rather than clean final generalization claims.

10 Reproducibility Appendix

This appendix is generated by the publication pipeline. It records the resolved inputs, availability checks, and release readiness for the current paper build.

10.1 Publication Readiness

- Required artifacts present: yes
- LaTeX build available on this machine: yes
- Hugging Face token present: no
- Birdeye key present: no

Table 7: Babylon scam-defense experiment log

Experiment	Samples	ScamBench	Generic	Status
bench v2 multit...	n/a	n/a	not run	benchmark...
baseline 7b base	n/a	78.50	not run	baseline
train v1 initial	40	63.00	not run	rejected
train v2 rebala...	48	80.50	not run	rejected
train v3 peak s...	48	98.00	2/3	peak
train v4 invali...	n/a	n/a	not run	invalid
train v5 balanced	60	80.50	3/3	balanced
train v6 narrow...	51	78.50	2/3	rejected
bench v7 qwen35...	n/a	83.32	not run	baseline
train v7 qwen35...	n/a	n/a	not run	invalid
train v8 qwen35...	23	85.11	0/3	ablation...
train v9 qwen35...	64	89.93	0/3	peak
bench v10 scamb...	n/a	n/a	not run	benchmark...
engine v10 baby...	n/a	n/a	not run	infrastru...
bench v11 qwen3...	n/a	75.33	not run	baseline
train v11 qwen3...	93	70.40	0/3	rejected
train v12 qwen3...	30	81.57	0/3	peak
train v13 qwen3...	104	80.17	0/3	balanced
bench v14 qwen3...	n/a	63.41	not run	baseline
train v14 qwen3...	30	62.86	0/3	ablation...
train v15 qwen3...	104	59.05	0/3	rejected
bench v16 herme...	n/a	84.99	not run	external...
data v17 extern...	n/a	n/a	not run	data review
data v18 extern...	n/a	n/a	not run	data pipe...
data v19 prompt...	n/a	n/a	not run	data pipe...
data v20 merged...	n/a	n/a	not run	data pipe...
data v21 awesom...	n/a	n/a	not run	data inve...
data v22 awesom...	n/a	n/a	not run	data mate...
data v23 difrau...	n/a	n/a	not run	data mate...
data v24 difrau...	n/a	n/a	not run	data synt...
model v24 difra...	n/a	n/a	not run	ablation
ops v25 release...	n/a	n/a	not run	release p...
release v1 qwen...	6,104	28.82	fail	peak
release v1 qwen...	8,126	24.46	fail	balanced
release v1 qwen...	6,104	18.69	fail	9b best
model v26 nebiu...	n/a	n/a	not run	ablation
remote v26 qwen...	n/a	8.49	not run	baseline
remote v26 qwen...	88	7.31	pass (action_reason)	optimizer...
remote v26 qwen...	515	7.31	pass (action_reason)	optimizer...
release v2 qwen...	88	10.89	pass (action_reason)	superseded
release v3 qwen...	110	8.21	pass (action_reason); json aux fail	corrected...
remote v26 qwen...	515	7.33	pass (action_reason)	optimizer...

Table 8: Resolved publication artifacts and input hashes

Artifact	Status	Requirement	SHA-256
babylon_matrix_manifest	found	optional	e7bb8d64604a
babylon_run_summary	found	optional	ae68a29d0115
babylon_export_manifest	found	optional	784ddcda164a
babylon_deployed_comparison	missing	optional	n/a
babylon_deployed_export_manifest	missing	optional	n/a
babylon_deployed_run_summary	missing	optional	n/a
local_training_manifest	missing	optional	n/a
tinker_training_manifest	found	optional	2f8dd8353f57
local_training_metrics	missing	optional	n/a
tinker_training_metrics	found	optional	079f3a44c596
local_training_validation	missing	optional	n/a
local_model_comparison	missing	optional	n/a
tinker_served_comparison	found	optional	68b0c2e38c43
tinker_benchmark_results	found	optional	f8a9cef38295
local_trustbench_baseline	found	optional	55a4faa02af5
local_trustbench_trained	found	optional	386fb98bf62e
hf_dataset_summary	found	optional	e382edf2cc01
hf_dataset_bundle	found	optional	n/a
social_trust_summary	found	optional	3ee31ae961d1
twitter_collection_manifest	missing	optional	n/a
twitter_collection_records	missing	optional	n/a
social_trust_leaderboard	found	optional	4bbfd3668d46
social_trust_timeline	found	optional	e728e771d86c
legacy_trust_rankings	found	optional	f37bdac031a0
legacy_trust_scores	found	optional	d84bbe9cf505
trust_bench_random	found	optional	da5ba79826d5
trust_bench_baseline	found	optional	f437fa2b6d3a
trust_bench_improved	found	optional	e7aea363af7e
scam_defense_experiments	found	optional	acf76f5b95ac
scambench_results	found	required	8cefc6600508
trust_sft_manifest	missing	optional	n/a
trust_rl_manifest	missing	optional	n/a
contrastive_analysis_report	missing	optional	n/a

10.2 Resolved Artifacts

10.3 Train/Eval Split Methodology

The export pipeline supports a grouped held-out split via `--held-out-ratio`. Entire scenario families (identified by the base scenario ID before any stage or repetition suffix) are deterministically assigned to either the training or evaluation partition using `SHA-256(seed:group_key) mod 1000`. This prevents any scenario-level contamination between train and eval.

Current release status. The release checkpoints (v1) were trained *before* the held-out split was implemented. Their train and eval exports overlap at the scenario level. The reported ScamBench numbers are therefore comparative ablations (baseline vs. trained under identical conditions), not held-out generalization measurements.

Reproducing with held-out split. To retrain with a proper split:

```
python scripts/export_scam_defense_trajectories.py \
  --held-out-ratio 0.15 --held-out-seed 42 \
  --include-format-recovery \
  --include-external-materialized
python scripts/train_local.py \
  --source-dir <export-dir> \
  --auto-detect-held-out
```

When `--auto-detect-held-out` is enabled, the trainer uses the `held-out/` subdirectory automatically for evaluation.

10.4 Format Recovery

The format-recovery multi-task path adds Action/Reason two-line exemplars to the training export covering all trading decision slices from the deterministic validation gate. The goal is to preserve the Babylon trading output format alongside defense supervision.

Current release status. The older reports that showed deterministic failure were overstating the issue because a prompt-prefill bug stripped `Action:` from otherwise valid responses before scoring. Under the corrected validator, the primary deterministic gate can pass even while the auxiliary JSON-format recovery check still fails. The multi-task path is now available via `--include-format-recovery` on the export script and `--format-recovery-dir` / `--format-recovery-ratio` on the trainer.

References

[1] G. A. Akerlof, "The market for "lemons": Quality uncertainty and the market mechanism*," *The Quarterly Journal of Economics*, vol. 84, pp. 488–500, 08 1970.

[2] P. Resnick, R. Zeckhauser, J. Swanson, and K. Lockwood, "The Value of Reputation on eBay: A Controlled Experiment," *SSRN*, Feb. 2003.

[3] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[4] E. Damiani, D. C. di Vimercati, S. Paraboschi, P. Samarati, and F. Violante, "A reputation-based approach for choosing reliable resources in peer-to-peer networks," in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, CCS '02, pp. 207–216, Association for Computing Machinery, 2002.

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.

[6] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" *arXiv preprint arXiv:2307.02483*, 2024.

[7] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. Available at https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[8] Anthropic, "Agentic misalignment: How LLMs could be insider threats," 2025. Available at https://www.anthropic.com/research/agentic-misalignment.

[9] Anthropic, "From shortcuts to sabotage: Natural emergent misalignment from reward hacking," 2025. Available at https://www.anthropic.com/research/emergent-misalignment-reward-hacking.

[10] Anthropic, "Sycophancy to subterfuge: Investigating reward tampering in language models," 2024. Available at https://www.anthropic.com/research/reward-tampering.

[11] E. Perez, S. Ringer, K. Lukosiute, K. Nguyen, E. Chen, S. Heiner, C. Pettit, C. Olsson, S. Kundu, S. Kadavath, *et al.*, "Red teaming language models with language models," *arXiv preprint arXiv:2202.03286*, 2022.

[12] J. B. Security, "Awesome prompt injection." GitHub repository. Available at https://github.com/Joe-B-Security/awesome-prompt-injection.

[13] elder-plinius, "L1B3RT4S." GitHub repository. Available at https://github.com/elder-plinius/L1B3RT4S.

[14] elder-plinius, "CL4R1T4S." GitHub repository. Available at https://github.com/elder-plinius/CL4R1T4S.

[15] elder-plinius, "OBLITERATUS." GitHub repository. Available at https://github.com/elder-plinius/OBLITERATUS.

[16] difraud, "difraud." Hugging Face dataset. Available at https://huggingface.co/datasets/difraud/difraud.

[17] H. Zhu, Z. Zhang, W. Cong, X. Liu, S. Park, V. Chandra, B. Long, and D. Z. Pan, "APOLLO: SGD-like memory, AdamW-level performance," *arXiv preprint arXiv:2412.05270*, 2024.

[18] A. Andriushchenko, A. Croce, and N. Flammarion, "AgentHarm: A benchmark for measuring harmfulness of LLM agents," in *ICLR*, 2025.

[19] A. Pan, J. S. Chan, A. Zou, N. Li, S. Basart, T. Woodside, J. Ng, H. Zhang, S. Emmons, and D. Hendrycks, "Do the rewards justify the means? Measuring trade-offs between rewards and ethical behavior in the MACHIAVELLI benchmark," in *ICML*, 2023.

[20] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, "R-Judge: Benchmarking safety risk awareness for LLM agents," in *EMNLP Findings*, 2024.

[21] H. Zhang, Z. Guo, K. Li, and C. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *ICLR*, 2025.

[22] Y. Yin, J. Zhong, and S. Cai, "SafeAgentBench: A benchmark for safe task planning of embodied LLM agents," *arXiv preprint arXiv:2412.13178*, 2024.

[23] T. Everett, C. Sherstan, and M. White, "ARLAS: Adversarial reinforcement learning for agent safety," *arXiv preprint arXiv:2510.05442*, 2025.

[24] S. Mehta, C. Zhao, and A. Madry, "ManagerBench: Measuring the safety-pragmatism tradeoff in LLM agents," *arXiv preprint arXiv:2510.00857*, 2025.

[25] X. Li, L. Wu, and J. Chen, "CNFinBench: A comprehensive benchmark for financial LLM safety and compliance," *arXiv preprint arXiv:2512.09506*, 2024.

[26] A. Bowyer, K. Sprague, and R. Kirk, "Don't use the CLT in LLM evals with fewer than a few hundred datapoints," in *ICML*, 2025.

[27] A. Jøsang and R. Ismail, "The beta reputation system," in *Proceedings of the 15th Bled Electronic Commerce Conference*, pp. 2502–2511, 2002.

[28] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.

[29] T. Yao, Z. Chen, and H. He, "The Logic Prior: Verifiable Rewards Improve LLM Reasoning Without Labeled Rationales," *arXiv preprint arXiv:2506.14245*, 2025.

[30] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, H. Cunningham, N. L. Turner, C. McDougall, M. MacDiarmid, C. D. Freeman, T. R. Sumers, E. Rees, J. Batson, A. Jermyn, S. Carter, C. Olah, and T. Henighan, "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," Anthropic, 2024.

[31] NVIDIA, "ProRL Agent: Rollout-as-a-Service for RL Training of Multi-Turn LLM Agents," *arXiv preprint arXiv:2603.18815*, 2025.

[32] Goedel-LM Team, "Goedel-Code-Prover: Hierarchical Proof Search for Open State-of-the-Art Code Verification," *arXiv preprint arXiv:2603.19329*, 2025.

[33] S. Tong, A. Mallen, D. Khashabi, and S. Singh, "What Makes a Conversation Persuasive? Identifying Factors of Conversational Persuasion through Multi-Turn Dialogue," *arXiv preprint arXiv:2409.20222*, 2024.